

PROGRAMAÇÃO DE
COMPUTADORES I

FUNÇÕES EM PYTHON

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

O PROBLEMA DO "COPIAR E COLAR"

O cenário: Você está **construindo** um sistema e percebe que precisa fazer a mesma **checagem várias vezes** ao longo do **código**.

- **Sistema Financeiro: Toda vez** que o operador **digita** um **valor** no caixa, você precisa de um **if** para **verificar** se o valor é **negativo** e **bloquear** a operação.

O PROBLEMA DO "COPIAR E COLAR"

O cenário: Você está **construindo** um sistema e percebe que precisa fazer a mesma **checagem várias vezes** ao longo do **código**.

- **Visual do Sistema:** Você precisa **imprimir** um **cabeçalho** bonitinho com o **nome** da empresa e uma **linha tracejada** (-----) antes de **mostrar** o **saldo**, antes de **mostrar** o **extrato** e antes de **sair**.

O PROBLEMA DO "COPIAR E COLAR"

A armadilha do iniciante: Selecionar o código, dar **Ctrl+C** e **Ctrl+V** **dezenas de vezes**.

Seu arquivo, que poderia ter **50** linhas, passa a ter **500**.

O EFEITO DOMINÓ DA MANUTENÇÃO

Copiar e colar parece rápido hoje, mas é um pesadelo amanhã.

- **Mecânica e Sensores:** Imagine o **software** de **injeção eletrônica** de um carro. Ele lê o **RPM** do motor em **vários momentos** (na partida, na aceleração, na frenagem) para checar se passou do **limite** de segurança.

O EFEITO DOMINÓ DA MANUTENÇÃO

Copiar e colar parece rápido hoje, mas é um pesadelo amanhã.

- **O problema:** Se a montadora decidir que o **limite de segurança** mudou de **6000** RPM para **5500** RPM, o programador terá que **procurar e corrigir** essa **regra** em **varios lugares diferentes** do código. Se esquecer um, o motor pode quebrar.

O MILAGRE DO PRINT()

Pense nos comandos que usamos desde a segunda aula: **print()** e **input()**.

- Para o **texto** aparecer na **tela**, o computador **precisa** converter texto em **binário**, **alocar memória**, mandar sinais **elétricos** para a **placa de vídeo** e acender os **pixels exatos** no seu monitor.

O MILAGRE DO PRINT()

Pense nos comandos que usamos desde a segunda aula: **print()** e **input()**.

- Você escreveu alguma dessas instruções elétricas complexas? Não.

O MILAGRE DO PRINT()

Pense nos comandos que usamos desde a segunda aula: **print()** e **input()**.

- Você apenas **digitou** `print()`. **Alguém** escreveu essa **lógica complexa** anos atrás, **empacotou** com um **nome simples** e **entregou** para você usar. Você usa a ferramenta sem precisar saber como o motor dela funciona por dentro.

O QUE É UMA FUNÇÃO?

A programação moderna baseia-se em **não reinventar a roda**.

- Uma **Função** é exatamente o que o **print()** é: uma "**mini-máquina**" ou **subprograma** que você **cria** dentro do seu **próprio código**.

O QUE É UMA FUNÇÃO?

A programação moderna baseia-se em **não reinventar a roda**.

1. Você **ensina** o **computador** a fazer uma **tarefa complexa** uma **única** vez, **empacota** essa tarefa e dá um **nome** a ela.
2. A **partir** desse momento, você só precisa **chamar** esse **nome** quantas vezes quiser.

POR QUE USAR FUNÇÕES?

Reaproveitamento Máximo: Escreva o **código uma vez, use mil** vezes. Seu arquivo fica drasticamente menor.

POR QUE USAR FUNÇÕES?

Manutenção Centralizada: Lembra do problema do **RPM** do motor? Com **funções**, você vai em um **único lugar** (dentro da função `chechar_rpm()`), **muda** o limite para 5500, e **todo** o resto do **sistema** é atualizado automaticamente.

POR QUE USAR FUNÇÕES?

Leitura Abstrata: O código principal passa a ser **lido** quase como um **manual em português**. Em vez de **blocos de matemática** complexa, você lê apenas nomes claros: **ler_sensores()**, **injetar_combustivel()**, **exibir_painel()**.

COMO ENSINAR NOVOS TRUQUES AO PYTHON

Para criar nossa **própria ferramenta**, usamos a palavra **def** (de definir).

```
1. def exibir_cabecalho():  
2.     print("=====")  
3.     print("  SISTEMA INICIADO  ")  
4.     print("=====")
```

COMO ENSINAR NOVOS TRUQUES AO PYTHON

Atenção: **Escrever** o **def** apenas **cria** a função. Para ela **funcionar**, você precisa "apertar o botão" **chamando** o nome dela no código:

```
1.exibir_cabecalho()
```

CASOS DE USO COMUNS

Fórmulas Matemáticas: Criar funções como **calcular_imc()** ou **converter_temperatura()**.
○ **programa principal** fica **limpo**, lidando apenas com a interação com o usuário.

CASOS DE USO COMUNS

Menus Visuais: Criar uma **função** **mostrar_menu()** para exibir opções (1. Cadastrar, 2. Sair) sempre que a **tela** precisar ser **atualizada**, sem **poluir** o código com **dezenas de prints**.

CASOS DE USO COMUNS

Fiscais de Validação: Criar funções de **segurança** como **validar_senha()** ou **verificar_idade()**. Elas **avaliam** a **entrada** e **dizem** se o **programa** pode **continuar** ou não, **atuando** como verdadeiros **seguranças** de porta isolados do resto do sistema.

ENTREGANDO "INGREDIENTES" PARA A MÁQUINA (PARÂMETROS)

Até agora, nossa função era **rígida**. Mas as **funções** podem ser **personalizadas**. Podemos **entregar dados** para elas **trabalharem**, os chamados **parâmetros** (**que colocamos dentro dos parênteses**).

SAUDAÇÃO

```
1. # Construindo a máquina (Ensinando a regra)
2. def dar_boas_vindas(nome_cliente):
3.     print("Olá,", nome_cliente, "! Sistema aberto!!")
4.     print("-----")
5.
6. # Usando a máquina (Apertando o botão)
7. dar_boas_vindas("Maria")
8. dar_boas_vindas("João")
```

A CALCULADORA DE IMC

```
1. # A Função (Nossa calculadora embutida)
2.
3. def exibir_imc(peso, altura):
4.     resultado_imc = peso / (altura ** 2)
5.     print("O IMC calculado é:", resultado_imc)
```

A CALCULADORA DE IMC

```
1.# O Programa Principal
2.peso = float(input("Digite o peso:"))
3.altura = float(input("Digite a altura:"))
4.
5.# Mandamos os dados para a função
   trabalhar:
6.exibir_imc(peso_digitado, altura_digitada)
```

O "FISCAL" DE REGRAS

```
1. def fiscalizar_valor(valor):  
2.     if valor < 0:  
3.         print("ALERTA!!!!!!")  
4.         print("Valor negativo!!!!!!")  
5.         print("Negado pelo Sistema.")  
6.     else:  
7.         print("Valor validado!!!!")  
8.         print("Operação permitida.")
```

O "FISCAL" DE REGRAS

1. # Testando o nosso fiscal em diferentes momentos:
2. `fiscalizar_valor(150)`
3. `fiscalizar_valor(-50)`

COMO A FUNÇÃO "FALA" COM O PROGRAMA?

Até agora, nós **enviamos dados** para dentro da **função** (os parâmetros), ela fez a **conta** e jogou o **resultado** na **tela** usando o **print()**.

Nós já nos comunicamos com as funções, mas como elas vão nos **responder**?

COMO A FUNÇÃO "FALA" COM O PROGRAMA?

O Problema do print(): O print é apenas um **letreiro**. Ele mostra o número para os **olhos humanos**, mas o computador "esquece" aquele valor logo em seguida.

COMO A FUNÇÃO "FALA" COM O PROGRAMA?

E se você quiser usar o **resultado** do **IMC** para fazer um **relatório**, **salvar** em um banco de dados ou usar em um **if** depois?

A função **não pode** apenas **imprimir** na tela, ela precisa **devolver** o valor matematicamente para o **programa principal**.

A RESPOSTA DA MÁQUINA

Para que a **função** entregue o **resultado** final de **volta** nas nossas **mãos**, substituímos o **print()** pela **palavra reservada return** (retornar/devolver).

Dica: Imagine o **print()** é a função falando com o humano e o **return** é a função falando com o computador.

O COMANDO RETURN

```
1. # A Função que calcula e DEVOLVE
2. def calcular_imcV2(peso, altura):
3.     resultado = peso / (altura ** 2)
4.
5.     return resultado
6. # A função entrega a resposta final
```

O COMANDO RETURN

```
1. # O Programa Principal
2. # Podemos guardar o valor que a função devolveu
3. imc_do_paciente = calcular_imcV2(80, 1.80)
4.
5. # Agora podemos usar a resposta
6.
7. if imc_do_paciente > 25:
8.     print("Atenção médico!!!!!!")
9.     print("O IMC do paciente é", imc_do_paciente)
10. else:
11.     print("Paciente dentro do peso ideal.")
```

DÚVIDAS???